

AI-based Fake News & Misinformation Detector

Domain: Social Media / Misinformation | AI/LLM Based Proof of Concept (POC)

1. Problem Statement

Pakistan mein social media (WhatsApp, Facebook, Twitter/X) par misinformation aur fake news bohat tezi se phailti hai, especially breaking news, political aur health-related topics par. Logon ke paas quickly verify karne ka koi asaan tareeqa nahi hota, jiske wajah se galat information viral ho jati hai aur real-world harm (panic, wrong health decisions) ka bais banti hai.

2. Proposed AI Solution

- User koi news text/headline/social media post paste karega system mein.
- LangChain few-shot prompting technique use karke LLM us text ko analyze karega — style, claims, aur language patterns dekh kar.
- System verdict dega: 'Likely Real', 'Likely Fake', ya 'Needs Verification' — sath mein reasoning bhi.
- Confidence score aur analysis history PostgreSQL mein save honggi, taake pattern track kiya ja sake.

3. Tools & Technologies

LangChain (few-shot prompting), OpenAI/Groq (LLM), FastAPI (backend), PostgreSQL + SQLAlchemy (history), Pydantic (structured verdict output), Streamlit (demo UI)

4. Dataset / Data Source

Sample labeled dataset — Kaggle par 'Fake and Real News Dataset' available hai, few-shot examples ke liye use hoga (training ke liye nahi, prompt examples ke liye).

5. Feasibility for POC

High — koi model training nahi chahiye, sirf achay few-shot examples aur prompt engineering se POC jaldi ban sakta hai.

6. Expected Impact

Log social media misinformation ko quickly verify kar sakege, panic aur galat decisions kam honge, aur digital literacy behtar hogi.

Technical Defense — Sir Q&A Preparation

7. Kya Use Karoon Gi (My Tech Stack) & Kyun

- Few-shot Prompting — LLM ko 3-5 labeled examples (real vs fake news patterns) dikha kar behtar classification karwaon gi, jaisa maine Day 19 mein practice kiya.
- LangChain chain (LCEL) — prompt + model + structured parser ka wahi pattern jo maine already seekha hai.
- Pydantic structured output — verdict (Real/Fake/Uncertain), confidence score, aur reasoning ko structured format mein force karoon gi.
- FastAPI + PostgreSQL — analysis history store karne ke liye, taake baad mein pattern/trends dekhe ja sakein.

8. Kya Use NAHI Karoon Gi & Kyun (Rejected Alternatives)

- Custom classification model training from scratch — bohat zyada labeled data aur time chahiye; LLM ka reasoning ability already achi hai zero/few-shot ke sath.
- 100% automatic 'fact-checking' claim — system sirf 'likelihood/pattern-based indicator' dega, final verification ke liye trusted fact-check sources ka link bhi doon gi, absolute truth ka claim nahi karoon gi.
- Real-time social media scraping (Twitter/Facebook API) — privacy aur API cost ki wajah se scope se bahar, POC mein user manually text paste karega.

9. Kaise Implement Karoon Gi (Step-by-Step)

- Step 1: Few-shot examples collect karna (Kaggle dataset se 5-10 clear real/fake examples select karke).
- Step 2: LangChain prompt template banana jismein examples + user ka input text ho.
- Step 3: LCEL chain build karna jo structured verdict (Pydantic model) return kare.
- Step 4: FastAPI endpoint jo text accept kare, chain call kare, result PostgreSQL mein save kare.
- Step 5: Streamlit UI jahan user text paste kare aur verdict + reasoning + confidence dekhe.

10. Possible Issues / Challenges & Mitigation

- LLM bias ho sakta hai kuch topics par — isliye verdict ko 'indicator' ki tarah present karoon gi, absolute fact nahi.
- Naye/breaking news jinke baray mein LLM ko pata nahi (knowledge cutoff) — unke liye system clearly 'insufficient information' boleگا.
- Satire/opinion pieces ko fake samajhne ka risk — isliye reasoning bhi show karoon gi taake user khud bhi judge kar sake.

11. Anticipated Questions from Sir & Answers

Q: Yeh system 100% accurate fake news detect karta hai?

A: Nahi, yeh ek pattern-based indicator hai jo language, claims aur style analyze karta hai; final verification ke liye trusted fact-checking sources recommend karoon gi, jaise sir clearly mention hoga disclaimer mein.

Q: Few-shot prompting kya hai aur yahan kyun use ki?

A: LLM ko 3-5 labeled examples dikha kar task samjhaya jata hai bina model retrain kiye — zero-shot se zyada accurate results deta hai, especially nuanced classification tasks mein.

Q: Agar news Urdu mein ho to system kaam karega?

A: Haan, modern LLMs multilingual hain; Urdu/Roman Urdu text bhi analyze kar sakte hain, bas few-shot examples mein kuch Urdu samples bhi add karoon gi accuracy behtar karne ke liye.

Project Architecture & Folder Structure

12. Folder Structure (Sir ke pattern par based)

```
fake_news_detector/
├── src/
│   ├── analysis/
│   │   ├── __init__.py
│   │   ├── models.py          # AnalysisRecord table (id, input_text, verdict, confidence, reasoning, created_at)
│   │   ├── schemas.py        # AnalyzeRequest, AnalyzeResponse
│   │   ├── router.py         # POST /analysis/check
│   │   ├── services.py       # few-shot prompt -> LLM classification logic
│   │   ├── utils.py          # text cleaning, source-credibility helper
│   │   ├── database.py
│   │   ├── settings.py
│   │   ├── main.py
│   │   └── prompts/
│   │       └── fewshot_examples.py # labeled example news (real/fake) for few-shot prompting
│   ├── logs/
│   │   └── app.log
│   ├── streamlit_app.py
│   ├── requirements.txt
│   ├── .env
│   ├── .gitignore
│   ├── notes.ipynb
│   └── README.md
```

13. Har File Mein Kya Likha Hoga

- **models.py** — SQLAlchemy table definition (e.g. analysis table: id, columns, timestamps).
- **schemas.py** — Pydantic models jo request body aur response ka shape/validation define karte hain.
- **router.py** — FastAPI endpoints yahan define hote hain — sirf request receive karke services ko call karta hai.
- **services.py** — Business logic: LLM chain call karna, DB query build/execute karna, result process karna.
- **utils.py** — Helper/reusable functions (e.g. prompt formatting, schema extraction, validation).
- **database.py** — SQLAlchemy engine, SessionLocal, aur Base declaration.
- **settings.py** — .env se DATABASE_URL, API keys load karta hai (Pydantic BaseSettings).
- **main.py** — FastAPI app instance, sab routers include karta hai, uvicorn isi ko run karta hai.
- **prompts/fewshot_examples.py** — labeled real/fake news examples ki Python list, jo prompt template mein dynamically inject hoti hai few-shot learning ke liye.

14. Data Flow — Sab Kaise Connect Hota Hai

- User Streamlit app mein news text/headline paste karta hai → request FastAPI ke router.py tak jati hai.
- router.py services.py ko call karta hai, jo prompts/fewshot_examples.py se examples le kar prompt banata hai.
- LangChain chain (prompt | LLM | Pydantic parser) verdict, confidence, aur reasoning generate karta hai.
- Result database.py ke through PostgreSQL ke analysis_records table mein save hota hai.
- Structured response schemas.py ke response model mein format hoke Streamlit ko wapis jata hai, jahan verdict aur reasoning dikhaya jata hai.

15. pgAdmin Mein Kya Data Show Hoga

- pgAdmin mein **analysis_records** table dikhega.
- Columns: id, input_text, verdict, confidence_score, reasoning, created_at.
- Query Tool se 'SELECT * FROM analysis_records;' run karne par sab past checked news aur unke verdicts history mein dikhenge.
- Yeh demo mein dikhaya ja sakta hai ke system kitni queries handle kar chuka hai aur verdicts ka distribution kaisa hai (kitni fake vs real flag hui).

16. Streamlit Interface Mein Kya Dikhega

- Streamlit app mein ek text area hoga 'Paste news text or headline here'.
- User text paste kar ke 'Analyze' button dabayega.
- Result: Badge/label ('Likely Real' / 'Likely Fake' / 'Needs Verification') color-coded (green/red/yellow), confidence % , aur neeche detailed reasoning paragraph.
- Optional: sidebar mein past analyzed items ki list (history).

17. uvicorn Chalane Par Kaunsi APIs Show Hongi (/docs)

- uvicorn src.main:app --reload chalane ke baad /docs par Swagger UI khulega.
- '/analysis' prefix ke andar: POST /analysis/check (text -> verdict+reasoning), GET /analysis/history, GET /health.
- 'Try it out' se POST /analysis/check test kiya ja sakta hai — {"text": "..."} body de kar turant verdict milega.
- /redoc par clean documentation view bhi available hoga.

18. Additional Anticipated Questions from Sir

Q: Few-shot examples kahan store honge aur kitne chahiye?

A: prompts/fewshot_examples.py ek Python file mein, 5-10 diverse examples (kuch clearly real, kuch clearly fake) — zyada examples se accuracy improve hoti hai lekin prompt length/cost bhi barhta hai.

Q: System kaise pata lagayega ke koi cheez satire hai ya fake news?

A: Yeh ek genuine limitation hai jo openly acknowledge karoon gi — reasoning output mein LLM apni uncertainty bhi express karega agar content ambiguous ho.

Q: Is system ko kaise improve kiya ja sakta hai future mein?

A: RAG add kar ke — trusted fact-check websites (jaise Snopes ya local fact-checkers) se real-time retrieval kar ke verdict ko aur grounded banaya ja sakta hai, jo future scope mein propose karoon gi.